



CarController.cs



Namespace

FougeraClub.Areas.Admin.Controllers



Class: CarController

Handles all CRUD operations, filtering, and reporting for `Car` entities in the Admin area.



Attributes

- `[AdminAuthorize]` — Restricts access to admin users.
- `[Area("Admin")]` — Specifies the area for routing.



Dependencies

Dependency	Purpose
<code>ICarService</code> <code>_CarService</code>	Service for car-related operations (CRUD).
<code>IUnitOfWork</code> <code>_unitOfWork</code>	Provides repository access to database tables.
<code>IMapper _mapper</code>	AutoMapper instance to map between <code>Car</code> entities and <code>CarVM</code> ViewModels.



Constructor

```
CarController(ICarService CarService, IUnitOfWork unitOfWork, IMapper mapper)
```

- Injects required services and mapper.
-

◆ Actions

1. `Index(...) : IActionResult`

- Returns the list of cars with optional filters: search term, type, and date range.
- Supports pagination and AJAX partial updates.
- Filters include:
 - PlateNumber, DriverName, Color, Model
 - Type filter
 - OwnershipExpiryDate range (`dateFrom`, `dateTo`)
- Sets `inService` flag based on `CarServices`.
- Returns a paginated `CarVM` list.

2. `AddEdit(int? id) : IActionResult`

- GET: Shows empty `CarVM` for adding or populated ViewModel for editing based on `id`.

3. `AddEdit(CarVM vm) : IActionResult`

- POST: Handles form submission for adding or editing a car.
- Validates file type and size for attachments.
- Maps `CarVM` to `Car` entity.
- Adds or updates the entity using `CarService`.
- Redirects appropriately after add or edit.

4. `Delete(int id) : IActionResult`

- Deletes a car by id using `CarService`.
- Also deletes the attached file from disk.

5. `Print(...) : IActionResult`

- Generates a print view of the filtered car list.
- Supports search term, type filter, and date range.
- Uses the same filtering logic as `Index`.

6. `createExcelReport_Download(...) : IActionResult`

- Generates an Excel report of the filtered car list.

- Supports both Arabic and English titles.
- Uses `ExcelStaticReport.ExcelReportArEn_` for report generation.
- Returns `FileContentResult` for download.
- Handles exceptions by redirecting to `Index`.

7. `GetCarAttachments(...)` : `IActionResult`

- Supports both Arabic and English names.
- Supports modal For Showing .
- Uses `<CarAttachmentsVM>` for server-side.
- All file uploads and deletions are managed via `FileHelper`.
- Get all this Car Attachments.

8. `UploadCarFiles (...)` : `IActionResult`

- Supports both Arabic and English names.
- Uses `<CarAttachmentsVM>` for server-side.
- All file uploads and deletions are managed via `FileHelper`.
- Upload all this Car Attachments and Remove.

Notes

- Uses `PaginatedList<CarVM>` for server-side pagination.
- AJAX support: Checks header `"X-Requested-With" == "XMLHttpRequest"` to return partial view.
- All file uploads and deletions are managed via `FileHelper`.
- Session language (`SessionHelper.GetCurrentLanguage()`) determines error messages and Excel report language.

Services



CarService.cs



Namespace

Application.Services.Admin



Class: CarService

Service class responsible for handling business logic and data operations for `Car` entities.
Implements `ICarService` interface.



Dependencies

- `IUnitOfWork _unitOfWork`: Repository abstraction for accessing database tables.
- `string FileName`: Folder name for storing car attachments (Cars).



Methods

Method	Parameters	Return Type	Description
<code>GetAllAsync()</code>	None	<code>Task<IEnumerable<Car>></code> >	Retrieves all cars from the database.
<code>GetByIdAsync(int id)</code>	<code>int id</code>	<code>Task<Car?></code>	Retrieves a specific car by its <code>Id</code> . Returns <code>null</code> if not found.
<code>GetAttachmentsAsync(int CarId)</code>	<code>int carId</code>	<code>Task<CarAttachmentsDTO></code>	Retrieves a specific car All Attachments related to it. Returns <code>null</code> if not found.
<code>UploadAttachmentsAsync(CarAttachmentsDTO model)</code>	<code>CarAttachmentsDTO model</code>	<code>Task<></code>	Save All uploads Attachments related to this car in Attachment table

Method	Parameters	Return Type	Description
<code>AddAsync(Car entity, IFormFile? file)</code>	<code>Car entity, IFormFile? file</code>	<code>Task<int></code>	Adds a new car to the database. If a file is provided, saves it using <code>FileHelper</code> and sets <code>AttachmentPath</code> . Returns the new car's Id.
<code>UpdateAsync(Car entity, IFormFile? file)</code>	<code>Car entity, IFormFile? file</code>	<code>Task</code>	Updates an existing car. If a new file is provided, deletes the old file and saves the new one. If no file is uploaded, keeps the old attachment.
<code>DeleteAsync(int id)</code>	<code>int id</code>	<code>Task</code>	Deletes a car by its Id. Also deletes the associated attachment file from disk.

Usage

- Provides CRUD operations for `Car` entities.
- Handles file management (saving and deleting attachments) transparently.
- Works with the `UnitOfWork` pattern to ensure database transactions are committed.

Notes

- File management is handled via `FileHelper`.
 - Ensures that existing files are deleted when updating or removing a car.
 - Designed to work with dependency injection in ASP.NET Core applications.
-

Entity

 **Car.cs**

 **Namespace**

Domain.Entities

Class: Car

Represents the `Car` entity in the domain model. This class is used to persist car information in the database.

Properties

Property	Type	Description
<code>Id</code>	<code>int</code>	Primary key of the car entity.
<code>PlateNumber</code>	<code>string?</code>	Car plate number. Maximum length 20 characters.
<code>Type</code>	<code>string?</code>	Type of the car (e.g., Sedan, SUV). Maximum length 100 characters.
<code>DriverName</code>	<code>string?</code>	Name of the driver assigned to the car. Maximum length 100 characters.
<code>OwnershipExpiryDate</code>	<code>DateOnly?</code>	Expiration date of car ownership. Stored as <code>date</code> type in the database.
<code>Model</code>	<code>string?</code>	Car model. Maximum length 100 characters.
<code>Color</code>	<code>string?</code>	Car color. Maximum length 100 characters.
<code>Notes</code>	<code>string?</code>	Additional notes about the car.

Property	Type	Description
<code>CarAttachments</code>	<code>ICollection<CarAttachment> ?</code>	Uploaded attachments files. <code>ICollection<CarAttachment></code> .

Usage

- Represents cars in the domain layer.
- Used by the database context to persist and query car information.
- Serves as the source entity for mapping to ViewModels like `CarVM` for the UI.

Notes

- No business logic or validation is included in the entity; validation is handled in the corresponding ViewModel (`CarVM`) or service layer.
- `CarAttachmentsVM` stores only the (file path,name); Uploaded attachments files. `ICollection<CarAttachment>`.

Views & UI Structure

File Structure

- `Index.cshtml` — Main page
- `_ListPartial.cshtml` — AJAX-loaded list
- `AddEdit.cshtml` — Create/Edit form
- `_CarAttachmentsModal.cshtml` — Modal form
- Models: `CarVM`, `PaginatedList`

Core Features

Vehicle Management

- Full fleet management
- Registration & documentation
- Ownership expiry tracking
- Rich vehicle metadata

Security & Permissions

- Role-based access control
 - Permission-based UI rendering
 - Delete validation against service dependencies
-



Filtering & Search System

Supported Filters

- Text search (PlateNumber, Model)
- Ownership expiry date range
- Vehicle type

Features

- Combined filter handling
 - Persistent state via **Session Storage**
 - Clear filter option
 - Real-time AJAX results
-



Export & Reporting



Print

- Uses applied filters
- 4000 record safety limit
- New window rendering



Excel Export

- Filter-aware export
 - Arabic / English output
 - Direct download
-



File Handling

- PDF-only uploads
 - Max size: **5MB**
 - Secure path handling
 - File preview in new tab
 - Safe deletion on update/remove
-

Technical Implementation

JavaScript

- AJAX partial reloads
- Print & Excel handlers
- Session storage persistence
- Pagination & tooltips

State Management

- Filter persistence
 - Dynamic UI updates
 - Partial rendering without reload
-

Integration Points

- Multilingual resources
 - Bootstrap UI & modals
 - jQuery AJAX
 - Validation scripts
 - FileHelper utilities
 - Car Service module integration
-

Responsive Design

- Mobile-friendly tables
 - Adaptive forms & buttons
 - Bootstrap grid system
 - Data-label support
-



Security Highlights

- PDF-only uploads
 - Size validation
 - Permission-based actions
 - Anti-forgery protection
 - Input sanitization
-



Vehicle Service Integration

- Add Service directly from car list
 - Dependency validation before delete
 - Permission-based service actions
 - Cross-module data consistency
-